TradeIQ (StockSense AI)

System Architecture, Quantitative Pipeline, Technology Stack & Implementation Overview

1. Project Overview

TradeIQ (StockSense AI) is a stock research and analysis web application developed as an engineering portfolio project. The system aggregates market data, computes deterministic technical indicators and fundamental valuation metrics, calculates statistical risk parameters (Annualized Volatility, VaR 95%, Sharpe, Beta, Max Drawdown), and runs rule-based backtests. Large language models (LLMs) via LangChain and Groq Cloud (primary: *qwen/qwen3.8-27b*; fallback: *openai/gpt-oss-20b*) are used to synthesize structured research memorandums.

2. System Architecture & Component Responsibilities

The application is structured into decoupled, modular tiers separating presentation, API routing, feature calculations, and AI synthesis:

Tier	Components & Libraries	Functional Responsibility
1. Presentation Layer	Vanilla HTML5, CSS Custom Properties, Modular ES6 JS, ApexCharts 7.1.0, Inter Font	Interactive user interface, candlestick and area charts, multi-step auth form with password manager pairing, and user preferences management.
2. API Gateway & Routing	FastAPI 0.135.2, Uvicorn ASGI 0.42.0, Pydantic V2 Schemas, SlowAPI Rate Limiter	Enforces ticker regex (`^[A-Z0-9.-]{1,10}$`), IP rate limits (5/min auth, 20/min data), JWT session tokens (60m), and process-time logging middleware. No CORS middleware.
3. Quantitative Analysis	Pandas 3.0.1, NumPy 2.2.6, indicator calculation routines (SMA 20/50/200, RSI-14, MACD, Volatility)	Ingests daily OHLCV price action via yfinance, computes moving averages, momentum oscillators, and historical volatility.
4. Sentiment & NLP	vaderSentiment 3.3.2, yfinance Search API, Yahoo Finance RSS Feed, Finviz Scraper	Retrieves recent company news headlines, computes VADER compound polarity scores (-1.0 to +1.0), and normalizes sentiment for report synthesis.
5. Risk & Strategy Backtesting	SciPy 1.17.1, NumPy 2.2.6, 252-day historical signal simulation	Calculates Annualized Volatility, Value at Risk (VaR 95%), Sharpe Ratio, Beta vs SPY, Maximum Drawdown, and non-overlapping trade signal win rates.
6. LLM Reasoning & Synthesis	LangChain 1.2.15, LangChain-Groq 1.1.2, qwen/qwen3.8-27b, openai/gpt-oss-20b	Synthesizes quantitative metrics into structured research memorandums. Note: Advisory memo scores are synthesized via LLM prompt; a separate deterministic weighted rule engine is in services/recommendation.py.

7. Persistence & Auth	SQLAlchemy 2.0.48 ORM, SQLite database, Passlib 1.7.4 (pbkdf2_sha256), Python-Jose 3.5.0	Stores user records with pbkdf2_sha256 password hashes, watchlists, and investment preferences (risk level, horizon, and analysis focus).

3. Verified Technology Stack Taxonomy

All dependency versions below match the project's requirements.txt specification and active environment:

Category	Technology / Package	Exact Version / Role
API Framework	FastAPI, Starlette	v0.135.2 — Async ASGI REST API endpoints
ASGI Server	Uvicorn	v0.42.0 — Production server with proxy headers
AI / LLM Orchestration	LangChain, LangChain-Groq, Groq	LangChain 1.2.15, Groq 0.37.1 (qwen/qwen3.8-27b)
Data Manipulation	Pandas, NumPy	Pandas 3.0.1, NumPy 2.2.6 — Convolutions, returns
Statistical Analysis	SciPy	v1.17.1 — Beta regression against SPY benchmark
Market Data Feed	yfinance, requests, beautifulsoup4	yfinance 1.2.0, bs4 4.14.3 — Price action & news
NLP Sentiment	vaderSentiment	v3.3.2 — Rule-based financial headline polarity
Database & ORM	SQLAlchemy	v2.0.48 — Relational models, SQLite database
Validation & Settings	Pydantic, pydantic-settings	pydantic 2.12.5, settings 2.14.0 — Strict validation
Security & Auth	Passlib (pbkdf2_sha256), Python-Jose	passlib 1.7.4, python-jose 3.5.0, slowapi 0.1.10
Frontend UI & Charts	Vanilla HTML5/CSS/JS, ApexCharts	Native ES6, ApexCharts 7.1.0, Inter typography
Containerization	Docker	Single-stage python:3.11-slim container on port 10000
Testing & Quality	Pytest, Pytest-Cov, Ruff	pytest 9.1.1, pytest-cov 7.1.0 (49 passing tests)

4. Key Implementation Details

• **Dual Scoring Architectures:** In the advisory module (services/ai_report.py), the research memorandum and scores are synthesized by prompting the LLM with quantitative evidence and user preferences. In services/recommendation.py, a deterministic rule-based heuristic calculates scores using fixed weights (35% Technical, 30% Fundamental, 25% Sentiment, with risk penalties).
• **Statistical Risk Modeling:** Calculates Annualized Volatility (daily log return std scaled by sqrt(252)), Value at Risk at 95% confidence (empirical 5th percentile), Beta vs S&P; 500 (ordinary least squares regression against SPY), Sharpe Ratio (assuming 5% risk-free rate), Maximum Drawdown, and Calmar Ratio.
• **Data Caching & Rate Resilience:** Implements an in-memory TTL cache (30 minutes) and exponential backoff retry loop (3 attempts) around yfinance fundamentals to withstand upstream rate limiting.
• **Password Security:** Uses passlib with the pbkdf2_sha256 algorithm. Password inputs are paired with username inputs inside standard form elements for password manager compatibility.

5. Known Limitations

• **Upstream Data Rate Limits:** Fundamental and historical pricing data depends on yfinance scraping public Yahoo Finance endpoints. Rate limits can occasionally lead to missing fundamental indicators.
• **Free-Tier Cold Starts:** When deployed on free-tier application hosting (e.g. Render free tier), the service spins down after inactivity, resulting in a cold-start delay on initial request.
• **LLM Response Variability:** While underlying statistical calculations are deterministic, narrative explanations generated by language models may vary across executions.
• **Database Concurrency:** Configured by default with SQLite, which is ideal for development and single-instance deployments but requires PostgreSQL for concurrent write scalability.

6. Quality Assurance & Verification

The project includes an automated test suite containing **49 passing unit and integration tests** covering technical indicator routines, fundamental parsing and caching, statistical risk formulas, historical backtesting, input validation, authentication, and user preferences. Code styling and formatting are verified using Ruff.

7. Disclaimer

NOT FINANCIAL ADVICE: TradeIQ (StockSense AI) is strictly an educational software project built for demonstration and research purposes. None of the indicators, ratings, memorandums, or backtests constitute investment, financial, or trading advice. Past performance is no guarantee of future results.